

Lab 1: De Struct a Clase

Explicación de la solución

1. El salto al paradigma orientado a objetos

En PRO1, definíamos un `struct` y después hacíamos funciones pasando la tupla como parámetro `funcion(tupla)`.

Ahora, se trata de hacer que estas “funciones” vivan **dentro** del `struct` (métodos), de manera que tengan **acceso directo al estado interno**. Cambiaremos la sintaxis a `tupla.funcion()`, encapsulando finalmente el código usando la protección que da `class`.

Cómo programábamos en PRO1 (fecha.cc)

```
“cpp [fecha.cc] struct Fecha { int dia, mes, año; };  
Fecha fecha_suma_dias(const Fecha& f, int dias) { Fecha res = f; res.dia += dias; // ... return res; }  
int main() { Fecha hoy = fecha_actual(); Fecha mañana = fecha_suma_dias(hoy, 1); }
```

Cómo programaremos en PRO2

Nuestra ``Fecha`` tiene ****responsabilidades propias****. Solo permite el acceso a variables y funciones públi

```
```cpp [main.cpp]  
class Fecha {
private:
 int dia, mes, año;

public:
 Fecha() { dia = 1; mes = 1; año = 2000; }

 Fecha suma_dias(int dias) const {
 Fecha resultado = *this;
 resultado.dia += dias;
 return resultado;
 }
};

int main() {
 Fecha hoy;
 Fecha mañana = hoy.suma_dias(1);
}
```

## Solución 1: Conversión a clase fecha.cc

[Simulació interactiva disponible a la web]

### Archivo original racional.cc

```
“cpp [racional.cc] struct Racional { int num_, den_; };

int mcd(int a, int b) { while (b != 0) { int aux = b; b = a % b; a = aux; } return a; }

int signo(int n) { return n < 0 ? -1 : 1; }

Racional racional_lee() { char ; Racional r; cin » r.num » _ » r.den_; return r; }

Racional racional_crea(int n, int d) { if (n == 0) { return {0, 1}; } int sig = signo(n) * signo(d); n = abs(n);
d = abs(d); const int m = mcd(n, d); return {sig * n / m, d / m}; }

Racional racional_suma(Racional a, Racional b) { return racional_crea(a.num_ * b.den_ + b.num_ *
a.den_, a.den_ * b.den_); }

Racional racional_resta(Racional a, Racional b) { return racional_crea(a.num_ * b.den_ - b.num_ * a.den_,
a.den_ * b.den_); }

Racional racional_multiplica(Racional a, Racional b) { return racional_crea(a.num_ * b.num_, a.den_ *
b.den_); }

Racional racional_divide(Racional a, Racional b) { return racional_crea(a.num_ * b.den_, a.den_ *
b.num_); }

void racional_escribe(Racional a) { cout « a.num_; if (a.den_ > 1) { cout « '/' « a.den_; } }

int main() { Racional acum = racional_lee(); racional_escribe(acum); cout « endl;

string op;
while (cin >> op) {
 Racional r = racional_lee();
 if (op == "+") {
 acum = racional_suma(acum, r);
 } else if (op == "-") {
 acum = racional_resta(acum, r);
 } else if (op == "*") {
 acum = racional_multiplica(acum, r);
 } else if (op == "/") {
 acum = racional_divide(acum, r);
 }
 racional_escribe(acum);
 cout << endl;
}

} “
```

## Solución 2: Conversión a clase racional.cc

[Simulació interactiva disponible a la web]